

Module 12: Microservices Architecture with Spring Boot – Eureka, Gateway, OpenFeign, and Resilience4J

Questions

Q1. What is a microservice architecture, and how is it different from a monolithic architecture?

Answer: A microservice architecture builds an application as a collection of small, independent services, each owning a specific business capability and its own data store. A monolithic architecture puts all functionality into a single deployable unit with a shared database. Microservices enable independent deployment, scaling, and technology choices, but they add network complexity, operational overhead, and distributed-system challenges. A monolith is simpler to build and test early on, but it becomes harder to scale and maintain as the codebase grows. The right choice depends on team size, domain complexity, and operational maturity, not on blindly following trends.

Q2. What are the advantages and disadvantages of microservices?

Answer: Advantages include independent deployment of services, scaling only the parts that need more resources, fault isolation so one failing service does not bring down the entire system, freedom to choose the best technology per service, and team autonomy around bounded contexts. Disadvantages include distributed data consistency challenges, network latency and failures, operational complexity for deployment and monitoring, the need for service discovery and API gateways, and harder end-to-end testing. In production, the biggest challenge is usually not writing the services but operating them consistently.

Q3. What is service discovery, and why is it needed in microservices?

Answer: Service discovery is the mechanism that lets services find each other's network locations at runtime instead of relying on hard-coded IP addresses and ports. In microservices, instances come and go due to scaling, deployment, and failures, so static configuration breaks quickly. A service registry tracks healthy instances, and clients or a gateway query it before making calls. Without service

discovery, every configuration change would require redeploying all consumers, which is not practical in a dynamic cloud environment.

Q4. What is Netflix Eureka, and how does it work?

Answer: Eureka is a service registry from Netflix, integrated into Spring Cloud through `spring-cloud-starter-netflix-eureka-server` and `spring-cloud-starter-netflix-eureka-client`. A Eureka Server runs a registry that accepts registrations from microservices. Each Eureka Client registers itself on startup and sends periodic heartbeats, usually every 30 seconds, to prove it is still healthy. If the server misses heartbeats for about 90 seconds, it removes the instance. Other clients can then fetch the registry and discover available instances by service name. Eureka supports client-side discovery, where the caller picks an instance, often with Spring Cloud LoadBalancer.

Q5. What is the difference between `@EnableEurekaClient` and `@EnableDiscoveryClient`?

Answer: `@EnableEurekaClient` is specific to Netflix Eureka and ties the application to that implementation. `@EnableDiscoveryClient` is a more generic annotation from Spring Cloud that works with any compatible discovery service such as Eureka, Consul, or ZooKeeper. If the classpath has a discovery implementation, `@EnableDiscoveryClient` auto-detects it. In modern Spring Cloud, the annotations are often unnecessary because auto-configuration detects Eureka on the classpath. However, understanding the difference matters when choosing between registries or when migrating from one to another.

Q6. What is client-side service discovery versus server-side service discovery?

Answer: In client-side discovery, the service client queries the registry directly, chooses an instance, and sends the request. Eureka with Spring Cloud LoadBalancer is client-side. In server-side discovery, the client sends a request to a load balancer or API gateway, and that intermediary queries the registry and forwards the request. Server-side discovery hides the registry details from clients and centralizes routing logic. Client-side discovery gives more control to the caller but couples it to the registry. Many architectures use both: a gateway handles server-side discovery from external clients, while internal services use client-side discovery through Feign.

Q7. What is an API Gateway, and what problems does it solve?

Answer: An API Gateway is a single entry point that sits between clients and backend microservices. It solves cross-cutting concerns such as request routing, authentication and authorization, SSL termination, rate limiting, load balancing, request and response transformation, caching, and logging. Instead of every service handling security and client-specific protocols, the gateway centralizes them. This reduces duplication and gives the backend services cleaner contracts. A common mistake is pushing business logic into the gateway; it should handle infrastructure concerns, not become another service layer.

Q8. What is Spring Cloud Gateway, and how is it different from Netflix Zuul?

Answer: Spring Cloud Gateway is the modern API Gateway in the Spring Cloud stack, built on Spring WebFlux, Project Reactor, and Netty, making it non-blocking and reactive. Netflix Zuul 1 was servlet-based and blocking. Spring Cloud Gateway supports route predicates, filter factories, load-balanced routing with `lb://`, rate limiting, circuit breaker integration, and actuator endpoints. It is the recommended replacement for Zuul, which is in maintenance mode. Because it is reactive, it can handle a large number of concurrent connections with fewer threads than a servlet-based gateway.

Q9. What are the three main building blocks of a Spring Cloud Gateway route?

Answer: A route in Spring Cloud Gateway consists of an ID, a destination URI, a collection of predicates, and a collection of filters. The predicate determines whether the route matches an incoming request. The filter modifies the request before it is proxied, the response before it is returned, or both. Multiple predicates on a route are combined with a logical AND, so all must match. The URI can be a direct HTTP URL, a load-balanced `lb://service-name`, or a `forward:` path for local fallback handling.

Q10. What are common route predicates in Spring Cloud Gateway?

Answer: Common predicates include `Path` for matching request paths, `Method` for HTTP methods, `Header` for request headers, `Query` for query parameters, `Cookie` for cookies, `Host` for host headers, `After`, `Before`, and `Between` for time-based routing, `RemoteAddr` for IP-based matching, and `Weight` for weighted traffic splits. For example, `Path=/orders/**` routes every request starting with `/orders/`. Predicates make it easy to implement blue-green deployments, A/B testing, and versioned routing without changing services.

Q11. What are common GatewayFilter factories in Spring Cloud Gateway?

Answer: Common filters include `AddRequestHeader` and `AddResponseHeader` for adding headers, `StripPrefix` for removing leading path segments, `PrefixPath` for prepending a path, `RewritePath` for regex-based path rewriting, `SetPath` for replacing the path, `SetStatus` for changing response status, `Retry` for retrying failed requests, `RequestRateLimiter` for rate limiting with Redis, `CircuitBreaker` for Resilience4J integration, and `FallbackHeaders` for passing exception details to a fallback. Default filters can also be configured so they apply to every route without repetition.

Q12. How does load balancing work in Spring Cloud Gateway?

Answer: Spring Cloud Gateway uses the `ReactiveLoadBalancerClientFilter` to resolve `lb://service-name` URIs. When a route uses `uri: lb://payment-service`, the gateway asks Spring Cloud LoadBalancer for a healthy instance of `payment-service` and forwards the request there. The default load balancing strategy is round-robin. If no instance is found, the gateway returns a 503 by default, but this can be changed to 404 with a property. This integration removes the need for clients to know instance locations and centralizes routing at the gateway.

Q13. What is OpenFeign, and why is it used in microservices?

Answer: OpenFeign is a declarative HTTP client that lets you define remote service calls as Java interfaces with Spring MVC annotations. Instead of writing boilerplate `RestTemplate` or `WebClient` code, you declare a method like `@GetMapping("/payments/{id}") Payment getPayment(@PathVariable Long id)` and Feign generates the implementation. It integrates with Eureka for service discovery, Spring Cloud LoadBalancer for client-side load balancing, and Resilience4J for fault tolerance. It reduces boilerplate and makes inter-service communication look like local method calls, though you must remember that network calls can still fail.

Q14. How do you enable and configure OpenFeign in Spring Boot?

Answer: You add `spring-cloud-starter-openfeign` to the classpath, annotate the main class or a configuration class with `@EnableFeignClients`, and declare an interface annotated with `@FeignClient`. The client name can be a service name registered in Eureka, in which case you omit the `url` attribute, or a fixed URL for direct calls. You configure timeouts, logging, and retry behavior in

`application.yml` under `spring.cloud.openfeign.client.config`. Without `@EnableFeignClients`, Feign interfaces are not scanned and no beans are created.

Q15. How does OpenFeign handle errors by default, and how can you customize it?

Answer: By default, Feign throws a generic `FeignException` for any non-2xx response, which makes it hard for callers to distinguish between client errors and server errors. To customize error handling, you implement an `ErrorDecoder` bean. Inside the decoder, you map HTTP status codes to domain exceptions, such as 404 to `NotFoundException` and 503 to `ServiceUnavailableException`. You can also return a `RetryableException` for transient failures so Feign's retry mechanism kicks in. The decoded exception can then be handled by a global `@RestControllerAdvice` or by the calling service.

Q16. How do you propagate headers such as authorization tokens or correlation IDs through OpenFeign?

Answer: You use a `RequestInterceptor` bean. The interceptor's `apply` method runs before every Feign request and can add, remove, or modify headers. For example, you can read the current JWT from `SecurityContextHolder` or a request attribute and set it as the `Authorization` header. You can make the interceptor global so it applies to all Feign clients, or client-specific by setting `configuration` on `@FeignClient`. This is essential for maintaining security context and distributed tracing context across service calls.

Q17. What is a circuit breaker, and why is it important in microservices?

Answer: A circuit breaker is a fault-tolerance pattern that prevents cascading failures when a downstream service is unhealthy. It monitors calls to a service and opens the circuit when failures exceed a threshold, causing subsequent calls to fail fast without consuming resources. After a cooldown period, it enters a half-open state and allows a limited number of probe calls. If those succeed, the circuit closes and normal operation resumes. Without circuit breakers, a slow or failing downstream service can exhaust thread pools and cause the entire system to collapse.

Q18. What are the states of a Resilience4J circuit breaker?

Answer: Resilience4J circuit breakers have three states. The CLOSED state is normal operation, where all calls pass through and outcomes are recorded in a sliding window. The OPEN state rejects calls immediately with a `CallNotPermittedException` when the failure rate or slow call rate exceeds a threshold. The HALF_OPEN state allows a configurable number of probe calls after a wait duration; if they succeed, the circuit closes, otherwise it opens again. The sliding window can be count-based or time-based, and it records successful, failed, ignored, and slow calls separately.

Q19. What is the difference between Resilience4J and Netflix Hystrix?

Answer: Netflix Hystrix is the older circuit breaker library and is now in maintenance mode, no longer supported in Spring Boot 3. Resilience4j is the modern, lightweight replacement designed for Java 8 and functional programming. It is modular, so you can use only the modules you need, such as circuit breaker, retry, rate limiter, bulkhead, and time limiter. Resilience4j integrates natively with Spring Boot 3, Micrometer, and Prometheus, while Hystrix did not. Resilience4j also supports both count-based and time-based sliding windows, which Hystrix lacked.

Q20. What Resilience4J fault-tolerance modules exist, and what does each do?

Answer: Resilience4J provides several modules. CircuitBreaker stops calls to a failing service to prevent cascading failures. Retry re-executes failed calls with configurable backoff. RateLimiter limits the number of calls per time window. Bulkhead limits concurrent calls to isolate resource usage, with semaphore-based and thread-pool-based variants. TimeLimiter enforces a timeout on asynchronous calls returning `CompletableFuture` or `CompletionStage`. These modules can be combined, and the recommended order is bulkhead first, then circuit breaker, then retry, then time limiter, so resources are protected before failures are retried.

Q21. What is the difference between a Resilience4J bulkhead and a rate limiter?

Answer: A bulkhead limits the number of concurrent calls to a resource, protecting against thread pool exhaustion and resource starvation. A rate limiter limits the number of calls within a time window, protecting against bursts and enforcing quotas. A bulkhead throws `BulkheadFullException` when the concurrency limit is reached. A rate limiter throws `RequestNotPermitted` when the rate is exceeded. They solve different problems and are often used together: the rate limiter prevents a caller from overwhelming a service, while the bulkhead ensures a failure in one area does not consume all available threads.

Q22. How do you configure Resilience4J in a Spring Boot application?

Answer: You add the `resilience4j-spring-boot3` or `resilience4j-spring-boot2` starter along with `spring-boot-starter-aop`, because Resilience4J uses Spring AOP for its annotations. You annotate methods with `@CircuitBreaker`, `@Retry`, `@RateLimiter`, `@Bulkhead`, or `@TimeLimiter`, and you provide configuration in `application.yml` under `resilience4j.circuitbreaker.instances`, `resilience4j.retry.instances`, and similar keys. Each instance has properties such as sliding window size, failure rate threshold, wait duration, and max attempts. The annotations also support a `fallbackMethod` that must have the same parameters plus an optional `Throwable` parameter.

Q23. What is an inventory management system in a microservices context, and how would you design it?

Answer: In a microservices context, an inventory management system owns product stock levels, reservations, and availability as an independent service. It exposes APIs to check stock, reserve items for orders, release reservations, and adjust stock after fulfillment. It typically has its own database and communicates with order and warehouse services via REST or events. Stock updates must be concurrency-safe, so operations often use optimistic locking with a version column or pessimistic locking for high-contention items. Events such as `StockReserved` or `StockReleased` are published to a message broker so other services can react asynchronously.

Q24. What is the Saga pattern, and how does it relate to distributed transactions in microservices?

Answer: The Saga pattern manages a long-running business transaction as a sequence of local transactions, each in its own service. If a step fails, the saga executes compensating transactions to undo earlier steps. There are two main styles. Choreography sagas use events: each service completes its work and publishes an event that triggers the next service. Orchestration sagas use a central coordinator that tells each service what to do and handles compensations. Sagas replace ACID transactions across services because distributed databases cannot easily guarantee isolation. The trade-off is eventual consistency and the complexity of compensation logic.

Scenario based questions

S1. A team wants to migrate a monolith to microservices. How do you decide where to start?

Answer: I start by identifying bounded contexts in the monolith using domain-driven design, looking for parts of the domain that change together, have different scaling needs, or belong to different teams. A good first candidate is a capability that is not on the critical path for every request, such as notifications, recommendations, or inventory, so failure during extraction does not stop the entire business. I also look for parts with high deployment conflict or scaling pressure, because those give the fastest return on investment. I avoid starting with the user authentication, order placement, or payment flow unless absolutely necessary, because those are usually the most coupled and risky. The migration strategy is usually the Strangler Fig pattern: put an API Gateway in front of the monolith, gradually route traffic to new services, and retire the old functionality once the new service is stable.

S2. A microservice cannot find another service at runtime. How do you debug service discovery?

Answer: I first check whether both services are registered in Eureka by looking at the Eureka dashboard or calling `/eureka/apps`. If the provider is missing, I verify that it has `spring-cloud-starter-netflix-eureka-client` on the classpath, that its application name is set, and that the Eureka Server URL is correct. I also check the heartbeat configuration, because if the instance does not send heartbeats, Eureka evicts it. If the consumer cannot discover the provider, I verify that the consumer fetches the registry and that the service name in the Feign client or load balancer matches the registered name exactly. Network issues such as security groups, wrong VPC, or DNS problems can also prevent discovery even when registration looks correct.

S3. A Spring Cloud Gateway route returns 503 even though the downstream service is healthy. What could be wrong?

Answer: A 503 from the gateway usually means no healthy instance was found for the load-balanced route. I check that the route URI uses the correct `lb://service-name` and that the service is registered in Eureka or the static load balancer configuration. I also verify that the service health endpoint returns 200 and that the service is not marked as `OUT_OF_SERVICE` in the registry. If the gateway is running in a different namespace or network, I check that it can reach the service instances. By default, Spring Cloud Gateway returns 503 when no instance is found; I can switch this to 404 if that fits the API contract. Additionally, the downstream service might be rejecting the gateway's request due to missing headers or path mismatch, so checking downstream logs is important.

S4. An OpenFeign client throws a generic `FeignException` for every downstream error. How do you improve error handling?

Answer: I implement a custom `ErrorDecoder` bean that maps HTTP status codes and response bodies to meaningful domain exceptions. For example, `404` becomes `ProductNotFoundException`, `400` becomes `BadRequestException`, and `503` becomes `ServiceUnavailableException`. For transient errors such as `502`, `503`, `504`, or `429`, I return a `RetryableException` so Feign's retry mechanism can try again. I also make sure the global exception handler maps these domain exceptions to appropriate HTTP responses so clients get useful error messages. Without this, every failure looks the same and the calling service cannot decide whether to retry, fail, or return a fallback.

S5. A circuit breaker opens too often during startup and trips normal calls. How do you tune it?

Answer: Startup can produce failures due to JVM warm-up, connection pool initialization, or dependent services still starting. I increase the `minimumNumberOfCalls` so the circuit does not open based on a small number of noisy failures. I also set a reasonable `slidingWindowSize` and use a time-based window if failures are bursty. The `failureRateThreshold` should reflect real unhealthy behavior, not normal startup variance. I add a warmup or readiness check so traffic is only routed after the service is truly ready. For slow-start dependencies, I increase `waitDurationInOpenState` or use a longer timeout. Monitoring the circuit breaker metrics with Micrometer helps distinguish between startup noise and genuine problems.

S6. A Resilience4J retry keeps retrying a failed payment call and creates duplicate payments. What is the issue?

Answer: Retrying non-idempotent operations like payments can create duplicates if the first attempt actually succeeded but the response was lost or delayed. The fix is to make payment operations idempotent by requiring a unique idempotency key from the client and rejecting duplicate keys on the payment service. The retry should only apply to idempotent operations or to transient failures where duplicates cannot occur. I also use the circuit breaker to stop retrying when the downstream is clearly unhealthy. In the retry configuration, I limit the number of attempts and use exponential backoff with jitter so retries do not all arrive at the same time. The most important rule is to retry only operations that are safe to repeat.

S7. An inventory service experiences race conditions when two orders try to reserve the same last item. How do you handle it?

Answer: I use optimistic locking with a `@Version` column on the inventory row. When two transactions read the same row and try to update it, the first update increments the version and succeeds, while the second update fails with an `OptimisticLockException` because the version has changed. The second order can then retry, check stock again, and either reserve or return an out-of-stock response. For extremely high-contention items, I switch to pessimistic locking with `LockModeType.PESSIMISTIC_WRITE` to serialize reservations, accepting the trade-off of lower concurrency. I also consider using a reservation queue or saga pattern so stock is reserved atomically as part of the order flow, and released if the order is canceled.

S8. A Spring Cloud Gateway route needs to rewrite the path before forwarding. How do you configure it?

Answer: I use the `RewritePath` or `StripPrefix` filter. For example, if external clients call `/api/v1/orders/123` but the downstream service expects `/orders/123`, I add `filters: - StripPrefix=1` to remove `/api/v1/`, or `filters: - RewritePath=/api/v1/(?<segment>.*), /${segment}` for more control. The rewrite happens at the gateway before the request is proxied, so the downstream service sees the path it expects. I test the route with actuator endpoints and with direct requests to make sure the rewrite works for all expected patterns, including trailing slashes and query strings. Path rewriting is common when versioning APIs at the gateway while keeping downstream paths stable.

S9. A gateway rate limiter blocks legitimate users after a traffic spike. How do you tune it?

Answer: I first check whether the rate limit is global or per user. A global limit is easy to set too low and can block all users during a spike. I switch to a per-user or per-API-key limit using a custom `KeyResolver`, or I use a token bucket algorithm that allows short bursts while maintaining a steady rate. I configure `replenishRate`, `burstCapacity`, and `requestedTokens` based on real traffic patterns rather than guesses. I also add a fallback or queue mechanism and monitor how often the limiter rejects requests. If the spike is expected, such as a sale event, I scale the backend and raise limits temporarily instead of permanently throttling users.

S10. A microservice calls another service synchronously, and the caller slows down when the callee is slow. How do you improve resilience?

Answer: I add timeouts so slow calls fail fast instead of holding threads. Then I add a circuit breaker around the call so repeated failures are detected and the circuit opens, returning a fallback or failing fast. I also add a bulkhead to limit how many concurrent calls can go to the downstream service, protecting the caller's resources. If the call does not need an immediate response, I switch to asynchronous messaging with Kafka or RabbitMQ, which decouples the services and removes the need for the caller to wait. The combination of timeout, circuit breaker, bulkhead, and asynchronous communication is the standard resilience toolkit for microservices.

S11. A Feign client works with Eureka in dev but fails in production with a connection error. What do you check?

Answer: I check the `url` attribute on `@FeignClient`. If a hard-coded URL is left from dev testing, it will not use Eureka in production. I also verify that the production profile does not disable Eureka, that the service name matches exactly, and that the Feign client is not using a different service discovery implementation. I check network connectivity between the caller and Eureka, and between the caller and the target service instances. If load balancing is failing, I inspect Spring Cloud LoadBalancer configuration and confirm that `spring.cloud.loadbalancer.ribbon.enabled` is not set to true in a way that conflicts with the modern load balancer.

S12. A gateway needs to authenticate requests before routing them. Where should the authentication logic go?

Answer: The gateway is a good place to terminate authentication and add security headers for downstream services. I use Spring Security in the gateway to validate JWT tokens or OAuth2 access tokens, and then I forward relevant claims to downstream services using filters such as `AddRequestHeader` or `TokenRelay`. The downstream services can then trust the headers added by the gateway but should still validate signatures or perform their own authorization checks. I avoid putting complex business authorization logic in the gateway; that belongs in the services. Keeping authentication at the gateway centralizes security and prevents every service from handling token parsing.

S13. A service-to-service call needs to pass a correlation ID for distributed tracing. How do you implement it with Feign?

Answer: I create a `RequestInterceptor` that reads the correlation ID from the current request's MDC or a request attribute and adds it as a header to every outgoing Feign request. The same interceptor also propagates the authorization token if needed. I make sure the receiving service extracts the correlation ID and adds it to its own MDC so logs across all services can be correlated. For asynchronous calls, I use a `TaskDecorator` to copy MDC values to new threads. If I am using Micrometer Tracing or OpenTelemetry, the propagation happens automatically through tracing instrumentation, but a custom Feign interceptor is still useful for legacy or non-standard headers.

S14. A microservices team is using synchronous REST calls everywhere, and deployments are tightly coupled. How do you reduce coupling?

Answer: I introduce asynchronous event-driven communication for operations that do not need an immediate response. For example, after an order is placed, the order service publishes an `OrderCreated` event to Kafka, and the payment, inventory, and notification services consume it independently. This decouples the order service from the availability of the other services and allows each service to be deployed independently. For operations that still require a response, I keep synchronous calls but add circuit breakers, fallbacks, and clear service contracts. The goal is not to eliminate synchronous calls entirely but to use events where eventual consistency is acceptable.

S15. A circuit breaker fallback method is not being invoked. What could be wrong?

Answer: I first check that the fallback method signature matches the original method exactly, including parameter types, and that it has an additional `Throwable` parameter at the end if the exception is needed. I verify that the method is public and in the same class as the annotated method. I also confirm that the call goes through the Spring proxy; if the method is called via self-invocation, the circuit breaker advice is bypassed. Another cause is that the exception is caught inside the method before it reaches the circuit breaker, or that the configuration for the circuit breaker instance is not loaded. I add tests that directly exercise the circuit breaker through the proxy and inspect the Resilience4J registry to confirm the instance exists.

S16. A Resilience4J bulkhead is configured, but under load the service still becomes unresponsive. Why?

Answer: A bulkhead protects only the operations it wraps. If other parts of the service consume all threads or if the bulkhead limit is set too high, the service can still become unresponsive. I check whether the bulkhead is semaphore-based or thread-pool-based; for blocking I/O, a thread-pool

bulkhead provides better isolation but requires the method to return `CompletableFuture`. I also verify that the bulkhead applies to the correct method and that calls go through the proxy. If the service is failing due to a different dependency, such as a database, the bulkhead around the HTTP client will not help. Proper monitoring of bulkhead metrics shows whether the limit is actually being hit.

S17. A microservice architecture needs centralized configuration. How do you handle it?

Answer: I use Spring Cloud Config Server backed by a Git repository to store environment-specific configuration. Each service fetches its configuration at startup from the config server, and profiles such as dev, staging, and prod are separated. I also use Spring Cloud Bus with a message broker to push configuration refresh events to all services without restarting them. Services expose `/actuator/refresh` to reload configuration on demand. Secrets are not stored in plain text in Git; instead, I use encrypted values with Spring Cloud Config encryption or store secrets in AWS Secrets Manager or HashiCorp Vault and reference them from the configuration. Centralized configuration prevents configuration drift and makes environments reproducible.

S18. An order service calls a payment service, an inventory service, and a notification service to complete an order. How do you manage consistency if one call fails?

Answer: I use the Saga pattern because a single ACID transaction across multiple services is not possible. In a choreography saga, the order service publishes events, and each service handles its own step and publishes the next event. If a step fails, a compensating event is published, such as `PaymentFailed` causing the order to be canceled and inventory reservation released. In an orchestration saga, a central orchestrator coordinates each step and invokes compensations directly. The orchestration style is easier to trace and reason about for complex flows, while choreography is more decoupled. I also make each local step idempotent so retries and compensations are safe.

S19. A microservice deployment causes a brief outage because dependent services are not ready. How do you handle startup ordering?

Answer: Microservices should not require a strict startup order, because services come and go dynamically. I design services to be resilient to missing dependencies at startup by using retries, circuit breakers, and fallbacks. In Kubernetes, I add readiness probes so traffic is not routed to a service until it can handle requests. Liveness probes restart unhealthy pods. I also use init containers or job hooks for one-time dependencies such as database migrations, but I avoid waiting for other services to start

because that creates fragile coupling. The correct approach is to make every service tolerate the absence of its peers temporarily.

S20. A gateway route needs to support blue-green deployment by routing 10 percent of traffic to the new version. How do you configure it?

Answer: I use the `Weight` predicate to split traffic between two routes that point to different service versions or instances. For example, one route with `Weight=group1, 9` points to the stable version, and another route with `Weight=group1, 1` points to the new version, giving a 90/10 split. Both routes can use the same path predicate. I monitor error rates and latency for the new version, and if it is healthy, I gradually shift the weights. This canary-style deployment through the gateway reduces the risk of rolling out a bad version to all users. I also ensure that both versions are registered under the same or different service names depending on the service discovery setup.

S21. A microservice exposes many small REST endpoints, and consumers complain about too many network calls. How do you reduce chattiness?

Answer: I introduce an API composition layer, either in the API Gateway or in a dedicated backend-for-frontend service, that aggregates multiple downstream calls into a single response. For example, a mobile app might need user profile, order history, and recommendations in one screen; instead of making three calls, it calls one composed endpoint. Another approach is to expand the domain boundaries so related data is returned together, though this must be done carefully to avoid creating a distributed monolith. Caching at the gateway or in the consumer also reduces repeated calls. The trade-off is that the composer becomes a new component to maintain, but it significantly improves client performance.

S22. A team wants to use gRPC instead of REST for internal communication. What are the trade-offs?

Answer: gRPC uses HTTP/2 and Protocol Buffers, which provides smaller payloads, faster serialization, and strong service contracts through `.proto` files. It supports bi-directional streaming and is excellent for low-latency internal communication. The trade-offs are that gRPC is less human-readable than JSON, harder to debug with simple tools, and not as universally supported by browsers and firewalls. Contract changes require careful schema evolution. For external-facing APIs, REST or GraphQL is usually better, while gRPC shines for high-throughput service-to-service communication. The choice should be based on latency requirements, team familiarity, and ecosystem compatibility.

S23. A service registry like Eureka is unavailable. How do services continue to communicate?

Answer: Eureka clients cache the registry locally and refresh it periodically. If the Eureka Server is unavailable, clients can still use their cached copy for a while. The refresh interval and eviction policy determine how long the cached copy remains useful. I also add health checks and retry logic so services can survive short registry outages. For longer outages, I consider a multi-node Eureka cluster for high availability, or a service mesh such as Istio that handles discovery independently. The key is that service discovery is a helper, not a runtime dependency for every single call, as long as clients cache and gracefully handle stale registry data.

S24. A gateway route with a circuit breaker filter forwards to a fallback controller. How do you pass the original failure details to the fallback?

Answer: I use the `FallbackHeaders` filter, which adds headers such as `Execution-Exception-Type`, `Execution-Exception-Message`, `Root-Cause-Exception-Type`, and `Root-Cause-Exception-Message` to the forwarded fallback request. The fallback controller reads these headers and can return a meaningful error response or log the original cause. Without this filter, the fallback receives only the request path and standard headers, making it hard to explain why the circuit opened. I also make sure the circuit breaker name and fallback URI are configured correctly in the route definition.

S25. A team is debating whether to use an API Gateway or direct service-to-service calls for internal traffic. What do you recommend?

Answer: For external client traffic, an API Gateway is almost always the right choice because it centralizes security, routing, and cross-cutting concerns. For internal service-to-service calls, direct calls through Feign or a service mesh are usually preferred because adding an extra gateway hop for every internal call increases latency and creates a single point of failure. However, in some organizations a dedicated internal gateway is used for policy enforcement and observability. My recommendation depends on the organization size and operational needs: small teams use a single gateway for external traffic and direct internal calls; large enterprises may add an internal gateway layer for governance.